Sender I/O: A Constructed Comparison

Document Number:	D4053R0
Date:	2026-03-13
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure
2. The Coroutine-Native Echo Server
3. Why I/O Results Are Different
4. Constructing the Sender Echo Server
5. Approach A1: Route Through `set_value`
   - What A1 Loses
6. Approach A2: Split Across Channels and Shared State
   - What A2 Loses
7. Approach B: Route Through `set_error`
   - What B Loses
8. Approach C: Decompose with `let_value`
   - Return-Type Constraint
   - What C Loses
9. The Trade-Off
   - When the Byte Count Determines Correctness
10. How Coroutines Achieve Structured Concurrency
11. Invitation
12. Acknowledgments

References

Abstract

Four sender-based TCP echo servers are constructed from the C++26 specification (P2300R10[1], P3552R3[2]) and compared against a coroutine-native echo server (Corosio[3]). Approach A1 routes I/O results through `set_value`, matching coroutine-native ergonomics but bypassing the channel-based composition algebra. Approach A2 routes the error code through `set_error` while capturing the byte count in shared state, restoring channel-based composition but removing the byte count from the completion signature and reintroducing shared mutable state. Approach B routes errors through `set_error`, retaining composition but converting every routine network event into an exception and discarding the byte count. Approach C uses `let_value` to decompose an A1-style compound result and re-route the error code to `set_error`, restoring channel-based composition without shared state or exceptions but discarding the byte count on the error path. No construction achieves both full data preservation and channel-based composition without shared state or exceptions. The authors invite any reader to construct a sender-based echo server that preserves compound I/O results, retains channel-based composition, and avoids exception round-trips, using C++26 facilities. The authors will incorporate any such construction into a future revision and re-evaluate every finding.

This paper is one of a suite of five that examines the relationship between compound I/O results and the sender three-channel model. The companion papers are D4050R0[15], "On Task Type Diversity"; D4054R0[7], "Two Error Models"; D4055R0[13], "Consuming Senders from Coroutine-Native Code"; and D4056R0[14], "Producing Senders from Coroutine-Native Code."

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed and maintain Corosio[3] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The findings in this paper are structural and hold regardless of which library implements the coroutine-native layer. The authors provide information, ask nothing, and serve at the pleasure

of the chair.

---

2. The Coroutine-Native Echo Server

Corosio[3] `do_session`:

```
capy::task<> do_session()
{
    for (;;)
    {
        auto [ec, n] = co_await sock_.read_some(
            capy::mutable_buffer(
                buf_, sizeof buf_));

        auto [wec, wn] = co_await capy::write(
            sock_, capy::const_buffer(buf_, n));

        if (wec || ec)
            break;
    }
    sock_.close();
}
```

Both values visible. No exceptions.

---

3. Why I/O Results Are Different

Infrastructure operations have binary outcomes:

Operation	Success	Failure
`malloc`	Block returned	Allocation failed
`fopen`	File handle returned	Open failed
`pthread_create`	Thread running	Creation failed
GPU kernel launch	Kernel running	Launch failed

Operation	Success	Failure
Timer arm	Timer armed	Resource limit

Every row is binary. The three channels map unambiguously.

I/O operations return compound results:

Operation	Result
`read`	`(status, bytes_transferred)`
`write`	`(status, bytes_written)`

Every OS delivers both values together.[8] Chris Kohlhoff identified the consequence for senders in P2430R0[5]:

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

Four approaches follow.

---

4. Constructing the Sender Echo Server

Each approach is constructed from P2300R10[1] and P3552R3[2].

Kühl enumerated five channel-routing options in P2762R2[10] Section 4.2 and noted: "some of the error cases may have been partial successes...using the set_error channel taking just one argument is somewhat limiting." Shoop identified the same difficulty in P2471R1[11]: completion tokens translating to senders "must use a heuristic to type-match the first arg."

P2300R10[1] Section 1.3 demonstrates the pattern:

```
return read_socket_async(socket, span{buff})
    | execution::let_value(
          [](error_code err,
             size_t bytes_read) {
              if (err != 0) {
                  // partial success
              } else {
                  // full success
              }
          });
```

Approach A1 piped into Approach C. The example does not show what happens to `bytes_read` when the error must reach `set_error`.

---

5. Approach A1: Route Through `set_value`

The I/O sender calls `set_value(error_code, size_t)` for all outcomes. Structured bindings work (P3552R3[2] Section 4.4):

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    for (;;)
    {
        auto [ec, n] = co_await async_read(
            sock, net::buffer(buf));

        auto [wec, wn] = co_await async_write(
            sock, net::const_buffer(buf, n));

        if (wec || ec)
            break;
    }
}
```

Nearly identical to Corosio. Both values visible. No exceptions.

What A1 Loses

The I/O sender never calls `set_error`. The composition algebra is bypassed:

- `when_all` does not cancel siblings on I/O failure - the failure arrived through `set_value`.
- `upon_error` is unreachable.
- `retry` does not fire.

The model reduces to one channel. `set_error` serves no purpose for I/O senders under A1.

---

6. Approach A2: Split Across Channels and Shared State

Route the error code through `set_error`; capture the byte count in shared state:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    for (;;)
    {
        std::size_t n = 0;

        co_await (
            async_read(sock, net::buffer(buf))
            | then([&](std::size_t bytes) {
                  n = bytes;
              })
            | upon_error(
                  [&](std::error_code ec) {
                      // ec visible, n stale
                  }));

        co_await (
            async_write(
                sock, net::const_buffer(buf, n))
            | then([&](std::size_t bytes) {
                  n = bytes;
              })
            | upon_error(
                  [&](std::error_code ec) {
                      // ec visible, n stale
                  }));
    }
}
```

All three channels in use. `upon_error` reachable. `when_all` cancels siblings. `retry` fires.

What A2 Loses

The byte count bypasses the channels:

- `retry` fires on `set_error` but the byte count is in `n`, not in the error channel.
- `upon_error` receives `error_code` alone. `n` reflects the previous stage, not the failed one.
- **Shared mutable state** across continuation boundaries - the hazard the sender model was designed to eliminate.

A1 with the error code moved to `set_error` and the byte count moved to shared state.

7. Approach B: Route Through `set_error`

`set_value(size_t)` on success, `set_error(error_code)` on failure. P3552R3[2] converts `set_error` to an exception via `AS-EXCEPT-PTR`. No non-throwing path:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    try {
        for (;;)
        {
            auto n = co_await async_read(
                sock, net::buffer(buf));

            co_await async_write(
                sock, net::const_buffer(buf, n));
        }
    } catch (std::system_error const& e) {
        // ECONNRESET, EPIPE, EOF arrive here
    }
}
```

What B Loses

- **Byte count.** 500 of 1,000 bytes written before `ECONNRESET` - gone.
- **Non-throwing path.** Every `ECONNRESET` requires `make_exception_ptr` + `rethrow_exception`.
- **Visible error path.** The error hides in `catch`, separated from the `co_await` site.

8. Approach C: Decompose with `let_value`

Pipe A1 into `let_value`, inspect both values, re-route the error code to `set_error`:

```
async_read(sock, net::buffer(buf))
    | let_value(
          [](std::error_code ec, std::size_t n) {
              if (ec)
                  return just_error(ec);
              return just(n);
          })
    | upon_error([](std::error_code ec) {
          // reachable
      });
```

The handler sees both values. Downstream, `upon_error` is reachable, `when_all` cancels siblings, `retry` fires.

Return-Type Constraint

`let_value` requires its callable to return a single sender type. `just(n)` and `just_error(ec)` are different types. The code above is simplified for exposition - the branches cannot be written as a simple `if` / `else` returning different sender types without additional machinery. The callable must return a type-erased sender, a variant sender, or use a helper that unifies the return type. This is an additional ergonomic and potentially runtime cost that Approaches A1, A2, and B do not incur.

What C Loses

`just_error(ec)` carries only the error code. `set_error` takes a single argument.

- `retry` never sees the byte count.
- `upon_error` receives `error_code` alone.
- Partial write. 500 of 1,000 bytes before `ECONNRESET` - gone once `just_error` emits.

C is a hybrid of A1 and B. The decomposition point moves to application code - an improvement. The byte count is still lost on the error path. The return-type constraint adds friction the coroutine-native approach does not have.

---

9. The Trade-Off

Each sender approach pays a cost the coroutine-native approach does not.

Property	A1 (`set_value`)	A2 (shared state)	B (`set_error`)	C (`let_value`)	Corosio
Error code at call site	Yes	Yes	No (in `catch`)	Yes (in handler)	Yes
Byte count at call site	Yes	Yes	No (discarded)	Yes (in handler)	Yes
Partial write preserved	Yes	Yes	No	No (on error path)	Yes
Byte count in completion sig	Yes	No (side state)	No (discarded)	No (on error path)	Yes (return value)
`when_all` cancels on I/O error	No	Yes	Yes	Yes	Yes (value-based)
Error handler reachable	No (`upon_error`)	Yes (`upon_error`)	Yes (`upon_error`)	Yes (`upon_error`)	Yes (`if (ec)`)
Retry on I/O error	No (`retry`)	Yes (`retry`)	Yes (`retry`)	Yes (`retry`)	Yes (`retry_after`)
Retry sees byte count	No	No	No	No	Yes
Exception on `ECONNRESET`	No	No	Yes	No	No
Shared mutable state required	No	Yes	No	No	No
Channels used for I/O	1 of 3	3 of 3	3 of 3	3 of 3	Values (no channels)

Infrastructure operations face no such trade-off. Their outcomes are binary.

The echo server is deliberately minimal. The compound-result problem is per-operation. Adding protocol complexity adds more call sites with the same trade-off, not a different one.

When the Byte Count Determines Correctness

Many HTTP servers - including Google's - skip TLS `close_notify`. The composed read returns `(stream_truncated, n)`. If `n` equals `Content-Length`, the body is complete and the truncation is harmless. If `n` is less, the body is incomplete. The byte count determines correctness.

Coroutine-native:

```
auto [ec, n] = co_await tls_read(
    stream, body_buf);
if (ec == stream_truncated
    && n == content_length)
    ec = {};  // body complete, ignore truncation
```

Under B, `set_error(stream_truncated)` arrives with no byte count. Under C, the `let_value` handler sees both values - but only if it has the HTTP framing context. A generic TLS adapter does not. Once `just_error(stream_truncated)` emits, the byte count is gone.

D4056R0[14] names this boundary the **abstraction floor**:

Region	What the code sees
Above the floor	`error_code` alone - composition works
Below the floor	`(error_code, size_t)` - both values intact

The coroutine-native model has no such boundary.

An HTTP/2 multiplexer issuing concurrent reads via `when_all` faces the same table at every I/O boundary.

---

10. How Coroutines Achieve Structured Concurrency

Capy[4] provides `when_all` and `when_any` as coroutine-native primitives. Both models provide structured concurrency. They differ in where the compound result is visible when the composition decision is made.

---

11. Invitation

Construct a sender-based echo server that:

- preserves compound I/O results on the error path,
- retains channel-based composition (`when_all` , `upon_error` , `retry`),
- avoids exception round-trips for routine error codes, and
- uses C++26 facilities (P2300R10[1], P3552R3[2], P3149R9[6]).

The authors will incorporate any such construction and re-evaluate every finding.

---

12. Acknowledgments

The authors thank Dietmar Kühl for the channel-routing enumeration in P2762R2[10] and for `beman::execution` , Chris Kohlhoff for identifying the partial-success problem in P2430R0[5], Kirk Shoop for the completion-token heuristic analysis in P2471R1[11], Peter Dimov for the refined channel mapping in P4007R0[9], "Senders and Coroutines," Michał Dominiak, Eric Niebler, and Lewis Baker for `std::execution` , Ian Petersen, Jessica Wong, and Kirk Shoop for `async_scope` , and Fabio Fracassi for P3570R2[12], "Optional variants in sender/receiver." The authors also thank Ville Voutilainen and Jens Maurer for reflector discussion, and Herb Sutter for identifying the need for constructed comparisons.

References

1. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
2. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
3. cppalliance/corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio
4. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy
5. P2430R0 - "Partial success scenarios with P2300" (Chris Kohlhoff, 2021). https://wg21.link/p2430r0
6. P3149R9 - " `async_scope` - Creating scopes for non-sequential concurrency" (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2025). https://wg21.link/p3149r9
7. D4054R0 - "Two Error Models" (Vinnie Falco, 2026). https://wg21.link/d4054r0

8. IEEE Std 1003.1-2024 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/

9. P4007R0 - ''Senders and Coroutines'' (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0

10. P2762R2 - ''Sender/Receiver Interface For Networking'' (Dietmar Kühl, 2023). https://wg21.link/p2762r2

11. P2471R1 - ''NetTS, ASIO and Sender Library Design Comparison'' (Kirk Shoop, 2021). https://wg21.link/p2471r1

12. P3570R2 - ''Optional variants in sender/receiver'' (Fabio Fracassi, 2025). https://wg21.link/p3570r2

13. D4055R0 - ''Consuming Senders from Coroutine-Native Code'' (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4055r0

14. D4056R0 - ''Producing Senders from Coroutine-Native Code'' (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/d4056r0

15. D4050R0 - ''On Task Type Diversity'' (Vinnie Falco, 2026). https://wg21.link/d4050r0